

Hooks do Claude Code

Guia de referência rápida

O que é: scripts que o Claude Code executa em pontos fixos do ciclo de vida. O modelo não decide se rodam — o harness executa. Instrução no prompt é *pedido*; hook é *lei*.

1. Onde configurar

Hooks vivem na chave `"hooks"` dos arquivos de settings (JSON **estrito** — sem comentários; arquivo inválido é rejeitado inteiro):

Arquivo	Escopo	Vai pro git?
<code>~/.claude/settings.json</code>	todos os seus projetos	—
<code>.claude/settings.json</code>	o projeto	sim
<code>.claude/settings.local.json</code>	o projeto, só sua máquina	não (gitignored)
plugin (<code>hooks/hooks.json</code>) / frontmatter de skill ou subagente	enquanto ativos	—

Precedência (mais específico vence): plugin > managed (org) > local > projeto > user. Mudanças nos settings são detectadas por file watcher (~1–3s) — sem reiniciar a sessão. Para desligar tudo temporariamente: `"disableAllHooks": true`.

Estrutura mínima (o aninhamento é grupo → `hooks[]`; o `matcher` fica no grupo):

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [
          { "type": "command", "command": "\\\"$CLAUDE_PROJECT_DIR\\\"/.claude/hooks/meu-hook.sh", "timeout": 10 }
        ]
      }
    ]
  }
}
```

2. Todos os eventos

`matcher` = filtro opcional no grupo; o que ele casa depende do evento (nome de tool, tipo, padrão — regex ou literais separados por `|`). `"—"` = dispara sempre.

Conversa

Evento	Dispara quando...	Matcher casa
<code>UserPromptSubmit</code>	usuário envia um prompt (antes do Claude processar)	—
<code>UserPromptExpansion</code>	um comando digitado expande em prompt	nome do comando/skill
<code>MessageDisplay</code>	texto do assistente está sendo exibido	—
<code>Stop</code>	Claude termina a resposta	—
<code>StopFailure</code>	o turno morre por erro de API	tipo do erro (<code>rate_limit</code> , <code>billing_error</code> ...)

Evento	Dispara quando...	Matcher casa
Notification	Claude Code emite notificação	tipo (<code>permission_prompt</code> , <code>idle_prompt</code> , <code>agent_completed</code> ...)

Tools e permissões

Evento	Dispara quando...	Matcher casa
PreToolUse	antes de uma tool executar	nome da tool (<code>Bash</code> , <code>Edit</code> , <code>mcp_*</code> ...)
PostToolUse	depois de uma tool ter sucesso	nome da tool
PostToolUseFailure	depois de uma tool falhar	nome da tool
PostToolBatch	um lote de tools paralelas termina	—
PermissionRequest	uma chamada precisa de decisão de permissão	nome da tool
PermissionDenied	o modo auto nega uma chamada	nome da tool

Sessão e contexto

Evento	Dispara quando...	Matcher casa
SessionStart	sessão inicia/retoma	<code>startup</code> , <code>resume</code> , <code>clear</code> , <code>compact</code> , <code>fork</code>
SessionEnd	sessão termina (orçamento de 1,5s p/ todos os hooks)	<code>clear</code> , <code>logout</code> , <code>other</code> ...
Setup	<code>--init-only</code> / <code>--init</code> / <code>--maintenance</code>	<code>init</code> , <code>maintenance</code>
PreCompact / PostCompact	antes/depois da compactação de contexto	<code>manual</code> , <code>auto</code>
InstructionsLoaded	um CLAUDE.md / rules é carregado	motivo do load
ConfigChange	um arquivo de config muda na sessão	origem (<code>user_settings</code> , <code>skills</code> ...)
CwdChanged	o diretório de trabalho muda	—
DirectoryAdded	diretório adicionado na sessão (<code>/add-dir</code>)	método
FileChanged	arquivo "vigiado" muda no disco	nomes de arquivo (<code>.envrc</code> <code>.env</code>)
WorktreeCreate / WorktreeRemove	worktree criado/removido	—

Subagentes, tasks e MCP

Evento	Dispara quando...	Matcher casa
SubagentStart / SubagentStop	subagente criado / finalizado	tipo do agente
TaskCreated / TaskCompleted	task criada / concluída	—
TeammateIdle	teammate de agent team ficando ocioso	—
Elicitation / ElicitationResult	servidor MCP pede input do usuário / usuário responde	nome do servidor MCP

3. O contrato do script

Entrada (stdin): JSON com `session_id` , `cwd` , `hook_event_name` + campos do evento (`prompt` no `UserPromptSubmit`; `tool_name` / `tool_input` no `PreToolUse`; `tool_output` no `PostToolUse`...).

Saída por exit code:

Exit	Efeito
0	sucesso; stdout JSON é interpretado. Texto puro no stdout: vai pro contexto do Claude em <code>UserPromptSubmit</code> / <code>SessionStart</code> ; nos demais, só pro log de debug
2	bloqueia a ação (<code>PreToolUse</code> , <code>PostToolUse</code> , <code>UserPromptExpansion</code> ...); o stderr vira o feedback. Em <code>SessionStart</code> / <code>Notification</code> não bloqueia — só mostra

Exit	Efeito
outros	se o stdout tiver JSON válido, o JSON decide; senão, erro não bloqueante

Saída por JSON (stdout) — campos universais:

Campo	Efeito
systemMessage	aviso visível ao usuário no transcript
continue: false + stopReason	para todo o processamento (raro — não é "log!")
hookSpecificOutput	envelope dos campos específicos do evento (leva hookEventName)

Por evento, os que mais importam:

Evento	Campo	Efeito
UserPromptSubmit	hookSpecificOutput.additionalContext	injeta contexto que o Claude vê junto do prompt (tem que estar DENTRO do envelope)
	hookSpecificOutput.decision: "block"	rejeita o prompt
PreToolUse	hookSpecificOutput.permissionDecision	allow / deny / ask (+ permissionDecisionReason como feedback)
	hookSpecificOutput.updatedInput	reescreve os argumentos da tool (é assim que se faz um proxy/rewriter)
PostToolUse / Stop	decision: "block" + reason	devolve feedback ao Claude e o faz continuar/corrigir
PermissionRequest	decision.behavior: "allow"	auto-aprova sem prompt
PermissionDenied	retry: true	permite retentar a chamada negada

4. Tipos de hook (além de command)

type	Quem trata	Uso típico
command	script shell (stdin/stdout)	o padrão — qualquer lógica
http	endpoint HTTP (POST do JSON)	integrar com serviço/CI
mcp_tool	tool de um servidor MCP conectado	reusar integrações MCP
prompt	uma chamada de LLM (single-turn, responde {"ok": true/false, "reason": ...})	validação semântica ("esse commit message está ok?")
agent	um subagente com tools (multi-turn)	verificação que exige ler arquivos

5. Campos de configuração úteis

- matcher — regex ou literais com | , no grupo (não no hook).
- if — regra estilo permissão (Bash(git *)) para refinar eventos de tool.
- timeout — em segundos. Defaults: 600s (command / http / mcp_tool), 30s (prompt), 60s (agent); UserPromptSubmit limita a 30s, MessageDisplay a 10s.
- statusMessage — texto do spinner enquanto o hook roda.
- args: [] — presente = execução direta sem shell (sem pipes/globs); ausente = via shell.
- once: true — (hooks de skill/subagente) roda uma vez por sessão.
- Variáveis: \$CLAUDE_PROJECT_DIR (raiz do projeto), \$CLAUDE_PLUGIN_ROOT , \$CLAUDE_ENV_FILE (recarregar env de Bash).

Vários hooks no mesmo evento: rodam em paralelo; na fusão dos resultados, o mais restritivo vence (deny > ask > allow); todos os additionalContext são preservados; dois hooks reescrevendo o mesmo updatedInput = ordem não determinística (evite).

6. Receitas prontas (da doc oficial)

Formatar código após cada edição — `PostToolUse`, matcher `Edit|Write`:

```
{ "type": "command", "command": "jq -r '.tool_input.file_path' | xargs npx prettier --write" }
```

Proteger arquivos (`.env`, locks) — `PreToolUse`, matcher `Edit|Write`, script que compara `tool_input.file_path` com padrões proibidos e sai com `exit 2` (stderr = motivo).

Notificação desktop quando o Claude precisa de você — `Notification`, matcher `permission_prompt`, comando `osascript -e 'display notification ...'` (macOS).

Reinjetar convenções após compactação — `SessionStart`, matcher `compact`, comando `echo 'Lembrete: use bun, não npm; rode os testes antes de commitar.'` (stdout vira contexto).

Auto-aprovar um prompt de permissão específico — `PermissionRequest` com matcher da tool, devolvendo `{"hookSpecificOutput": {"hookEventName": "PermissionRequest", "decision": {"behavior": "allow"}}}`.

Auditar mudanças de config — `ConfigChange` com `jq` anexando `{timestamp, source, file}` num log.

7. Armadilhas clássicas

1. **Arquivo errado:** não existe `.claude/hooks.json` de projeto (isso é formato de plugin) — hooks de projeto vão em `settings.json`.
2. **Evento inventado:** os nomes são fixos, PascalCase (é `UserPromptSubmit`, não `beforeSubmitPrompt`).
3. **Estrutura achatada:** o formato é `hooks.<Evento>[].hooks[]` com `"type": "command"`.
4. **`continue: false` como "log":** isso para TODO o processamento; para observar use `systemMessage` / `additionalContext`, para bloquear use `decision: "block"` / `exit 2`.
5. **Comentários no `settings.json`:** JSON estrito; o arquivo inteiro é rejeitado (aparece no startup e no `claude doctor`).
6. **`additionalContext` no top-level** em `UserPromptSubmit`: precisa estar dentro de `hookSpecificOutput`.
7. **Esquecer o `chmod +x`** do script e caminhos relativos: use `$CLAUDE_PROJECT_DIR`.

Fontes (doc oficial): [hooks-guide](#) · [hooks \(reference\)](#) · [settings](#) · [env-vars](#). A lista de eventos cresce a cada versão — confira a reference para o estado atual.